Vanderbilt School of Engineering
Department of Mechanical Engineering
ME 4271: Final Project Report
Victoria You and Seth Hemingway

4a. Developing a GUI and Simulation for MECA500 (R3) Control Using Resolved Rates

December 7, 2023

Introduction

The purpose of this project is to demonstrate controlling Mecademic's MECA500 robot using a custom graphical user interface (GUI) and implementing a resolved rates algorithm for control. The robot itself is a 6-DOF serial robot with frame designations shown in Fig. 1. The world frame (WRF) naturally aligns with the base frame (BRF) before any manual changes, so, for clarity, any instance of the WRF can be thought of as the BRF. The robot also comes with a gripper attachment for an end-effector which can be utilized for pick-and-place objectives.

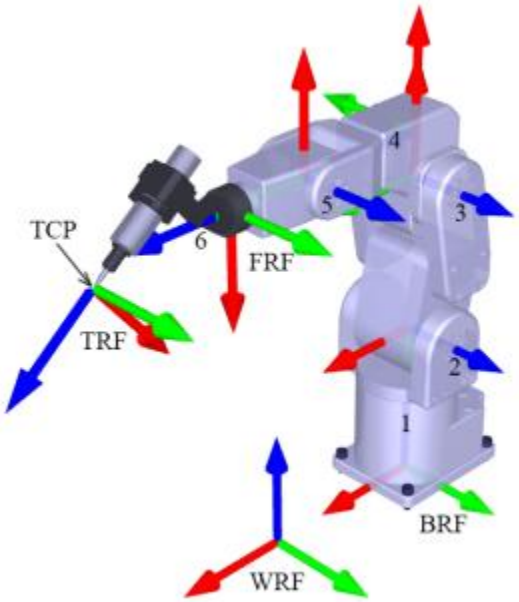

Figure 1: MECA500 Frame Assignment

Control of the robot was achieved using a TCP/IP communication protocol in conjunction with MATLAB's App Designer. Together they allow one to manually control the robot with various functions (a list of which can be found in the MECA500's programming guide). The exact functions involving the setup and ability to control the robot were built using object-oriented MATLAB functions that sent executables to the robot. Once a connection was established, user input via a MATLAB GUI allowed for complete control of end-effector position and joint values, as well as other miscellaneous abilities such as gripper control and error handling. By utilizing a resolved rates algorithm involving path-planning and singularity/collision avoidance, control of the robot was simulated and demonstrated physically.

Results

Our resolved rates algorithm was written in MATLAB, based on Dr. Simaan's lecture notes, with the intent on controlling both the simulation and physical implementation of the robot given initial joint values and a desired final pose. Since it may be difficult to know if a final pose

is achievable, the final pose can also be given as final joint values, but these values are converted to pose values using a numerical inverse kinematics calculation before running the resolved rates algorithm. The simulation implements the MecaVisualizer class provided with HW4, and the physical robot control implements MECA's built in control functions.

The algorithm begins with a setup declaring variables for constant values (joint bounds, end-effector and joint speed limits, and maximum errors). These variables are described in greater detail within the code comments, which has been copied into the Appendix. The function then calculates the initial joint and pose values, as well as the pose error before beginning the actual resolved rates loop. All values are relative to the world frame.

The position error is quite simple—a vector subtraction of $P_E = P_D - P_C$, where P_D is the desired position vector and P_C is the current. The rotation error is calculated as a rotation matrix defined as $R_E = R_D(R_C)^T$, where R_D is the desired rotation matrix and R_C is the current. Both errors are then represented as scalars—position as the normalized P_E vector and rotation as the angle[rad] from the angle-axis parameterization of R_E.

The resolved rates loop itself is a while loop running while the position and rotation errors are each greater than their respective epsilon values—maximum error allowed. Using the scalar representation of P_E and R_E, the linear and angular velocities (v and ω, respectively) of the end-effector are adjusted to move at a maximum speed at large error values and slow at small error values. This helps minimize pose overshooting while preventing extreme speeds and convergence times.

The instantaneous twist, $\dot{x}$, is then described as [v; ω] in a 6x1 matrix. By the definition of the Jacobian, we can then solve for our instantaneous joint speeds: q_dot = pinv(J)*$\dot{x}$. The Jacobian has its own function, getJacobian, which uses the DH table of the MECA and the definition of the geometric Jacobian provided in lecture notes.

The values of q_dot are then checked and adjusted to ensure the maximum joint velocities are not exceeded. The joint values are then updated with q = q + q_dot*dt, with each value being checked for violation of joint bounds prior to movement. The current pose and pose error variables are then updated using direct kinematics with the DH table, and the loop continues.

This algorithm seemed to work quite smoothly for both the simulation and physical implementation. They both apply the same algorithm, but with minor adjustments to fit their frame assignments and respective abilities to gather and send information. For example, the simulation automatically records a video using VideoWriter, which is unnecessary in the physical implementation. However, the bigger difference comes with the DH tables and frame assignments.

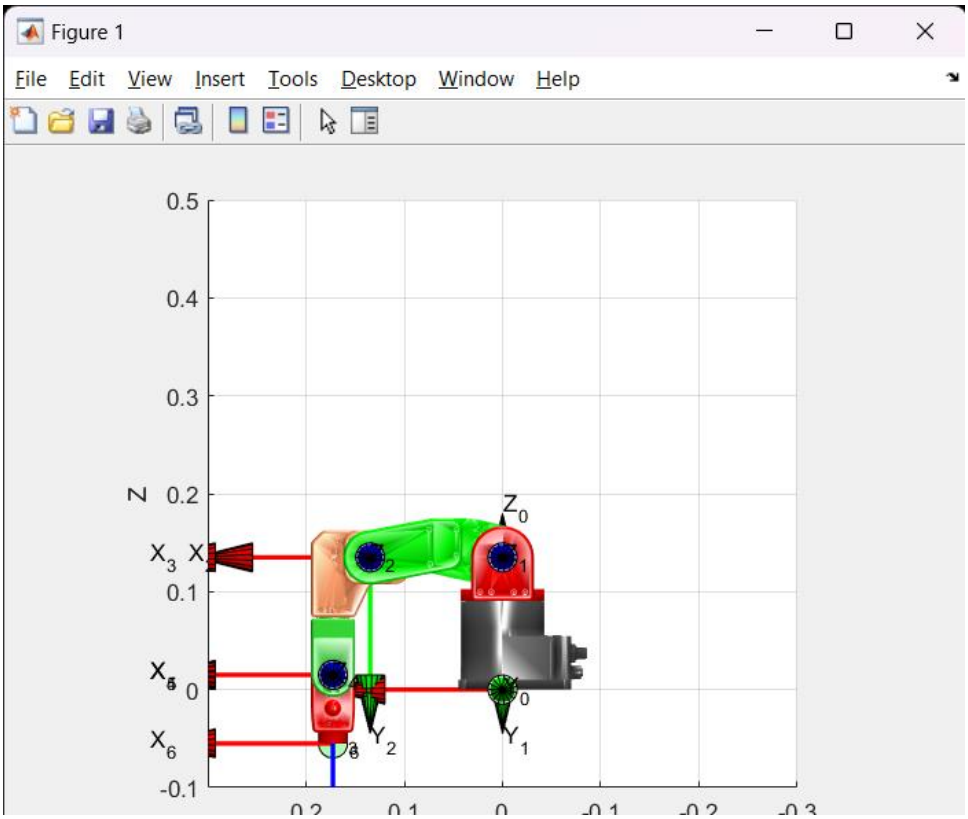

Figure 2: Zero Position of Simulation MECA

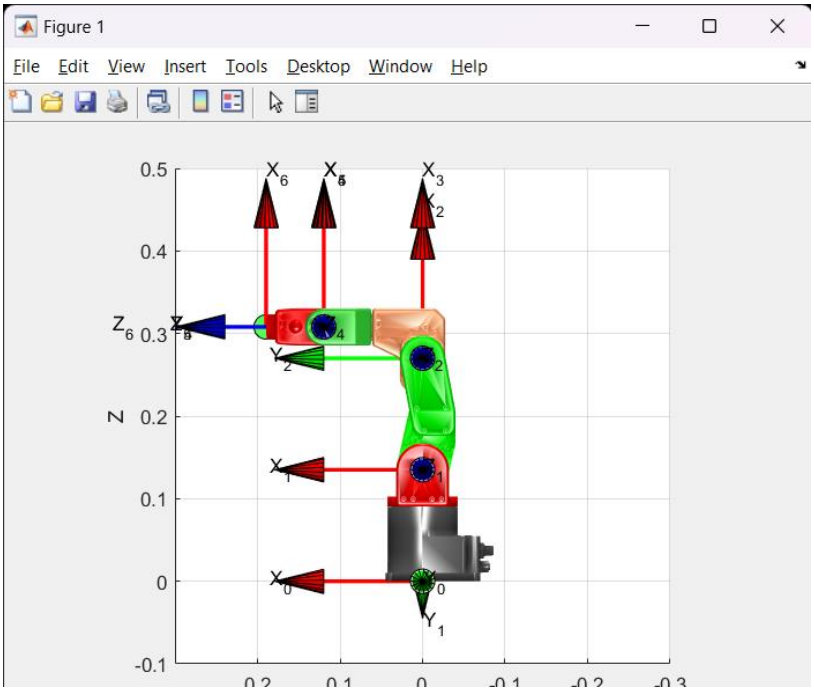

Figure 3: Zero Position of Physical MECA

Figure 2 displays the simulated "zero position" of the MECA while Figure 3 displays the physical implementation. The difference comes from the DH tables- $\theta_2 = q_2$ in the simulated MECA object's DH table, while $\theta_2 = q_2 - 90°$ in the physical MECA object's DH table. For uniformity, we adjusted the values in the simulation code to align with the physical implementation, but this makes the implementations slightly different.

The user interface shown in Figure 4 was implemented for physical control along with some helper functions for calculations during complicated movement operations. It contains 9 push buttons and 9 edit boxes that are enabled at different points during implementation. Only allowing certain parts of the GUI to be interacted with at a given time limits the risk of errors with movement queue execution and signal communication. The functionality and availability of each interactive button is described in Table 1.

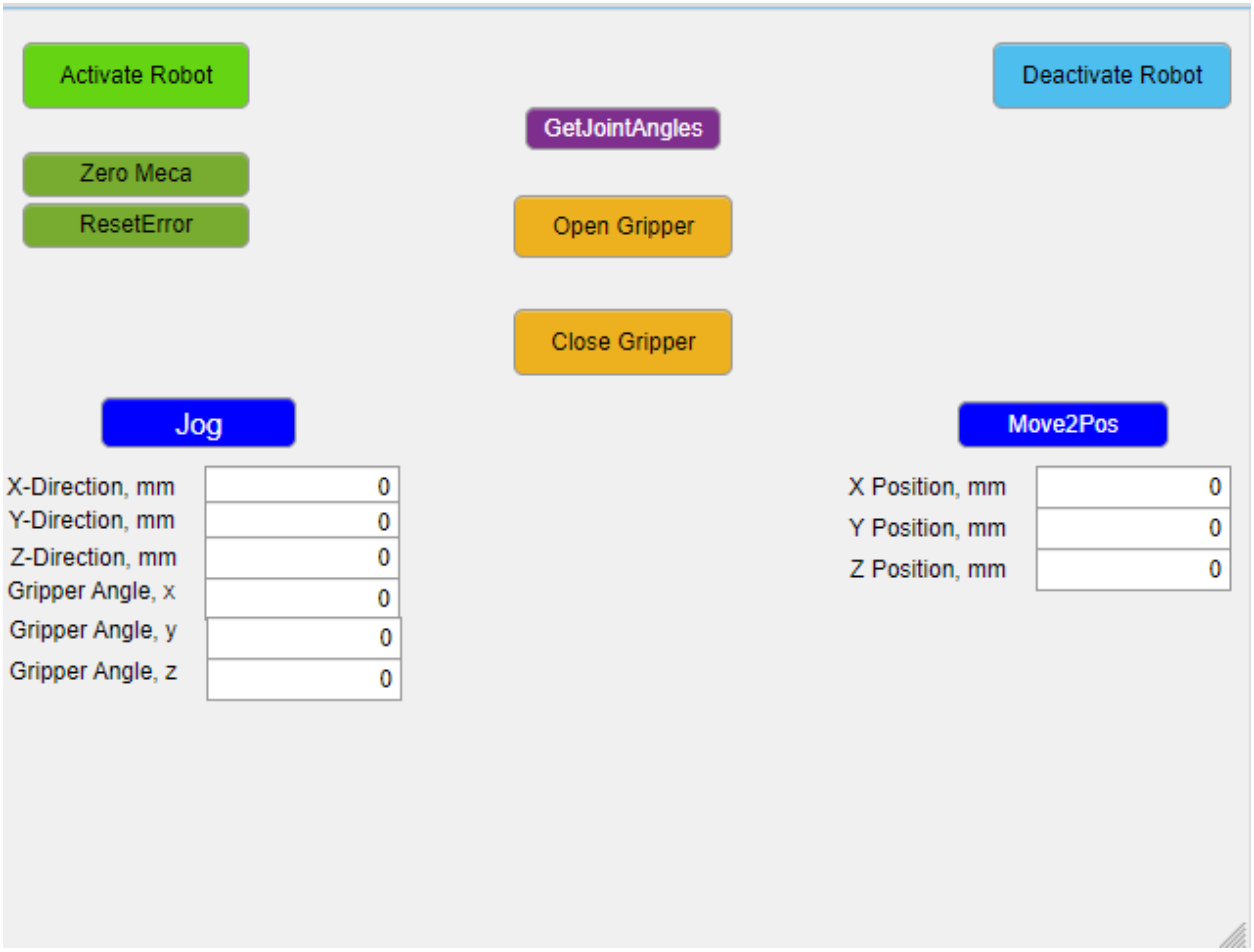

Figure 4: MATLAB GUI for MECA500 Control

Button Name	When it is Available	Functionality
Activate Robot	On startup	-Creates a connection between the host computer and robot -Homes the robot -Enables all other buttons
Zero Meca	After “Activate Robot” Callback	-Moves the end effector to a set position, helpful when an objective is completed
ResetError	After “Activate Robot” Callback	-If the robot enters an error state, press to allow movement
Jog	After “Activate Robot” Callback	-Allows one to move the end effector relative to a frame at the end effector and parallel to the world frame as determined by edit boxes
Move2Pos	After “Activate Robot” Callback	-Allows one to move the end effector to a pose defined from world frame position and Euler angles determined by edit boxes
GetJointAngles	After “Activate Robot” Callback	Returns the joint values in degrees as well as a timestamp
Open Gripper	After “Activate Robot” Callback	Opens the gripper
Close Gripper	After “Open Gripper” Callback	Closes the gripper to a grip force of 40N
Deactivate Robot	After “Activate Robot” Callback	-Terminates the connection between the host computer and robot -Deactivates all other buttons until “Activate Button” is called again

Table 1: GUI Functionality

Discussion and Conclusion

When the path was simple and end effector orientation non-tortuous, the resolved rates algorithm worked very well. However, there were many different situations where a simple error correction approach couldn’t achieve the desired results. These singularities were largely the result of our end-effector position converging before our orientation. Once our position converged, the resolved rates would try to keep it in place while correcting orientation, which was often impossible. This was due to singularities, joint collisions, and joint limits. Singularities

occur when the robot loses a degree of freedom, and the MECA500 has three types, shoulder, elbow, and wrist, examples of which can be found in the programming manual and seen below.

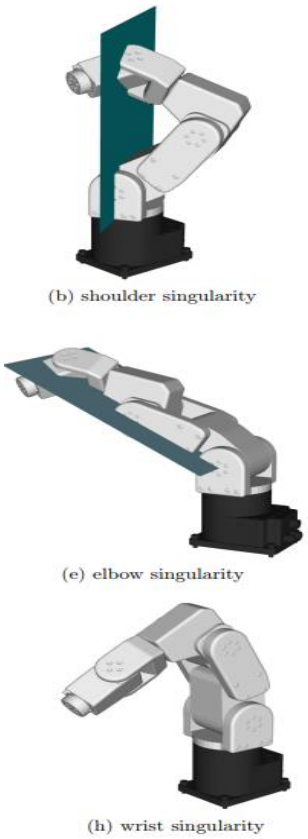

Figure 5: Examples of Robot Singularities

While elbow singularities, as well as many joint limits, can generally be avoided by not approaching the workspace boundary or by preventing impossible user inputs, we needed to adjust our instantaneous inverse kinematics to adjust for the other types. By monitoring the determinant of the instantaneous Jacobian, which approaches zero as it nears a singularity, we can detect when we need to offset the end effector, which we do by changing the joint angles of the last three joints seeing as the majority of singularities we encountered were wrist singularities.

Once we do so, the resolved rates algorithm continues from the offset position. This solution is mainly applied to the simulation, as the physical implementation isn't hindered by singular positions so long as the following movement is possible. This is done using the q_offset function, which imitates the Meca's "MoveJoints" function with a negligible error. This same approach can be applied to the joint limits, which are displayed in Table 2. If a joint limit was hit, the joint could simply rotate and continue from the new position. So long as a pose is attainable, this solution generally works.

Range for joint 1	$[-175°, 175°]$
Range for joint 2	$[-70°, 90°]$
Range for joint 3	$[-135°, 70°]$
Range for joint 4	$[-170°, 170°]$
Range for joint 5	$[-115°, 115°]$
Range for joint 6	$[-36,000°, 36,000°]$

Table 1: MECA500 Joint Limits

The final challenge to overcome was potential collisions. Since our resolved rates algorithm computes instantaneous inverse kinematics that force the robot to take the fastest path to an end pose, one can easily conclude that if the end effector started directly to the right of the robot and wanted to go to a parallel point on the other side, the algorithm would, initially, try to move the end effector through the robot. While the robot will automatically stop when a potential collision is detected, we wanted to add our own solution for two reasons: Our simulation did not have this feature, and we wanted the robot to continue its path rather than stop completely.

Looking at the frame assignment in Figure 1, it's clear that a situation like this requires the end effector position to either flip the sign of the x or the y position. Knowing that, we could catch this inversion before beginning the movement loop and move the end-effector to a predetermined position that would avoid this issue.

While solutions to these challenges may not have been the most computationally or physically efficient, they proved to solve many path planning problems in both the simulation and physical implementation.

Appendix

This appendix contains links to instructions on robot operation as well as the code used to do resolved rates calculations in both simulations and physical implementations. The GUI was created using MATLAB AppDesigner, the code for which can be found in the project folder along with all helper functions called.

MECA500 User Manual:
https://cdn.mecademic.com/uploads/docs/meca500-r3-user-manual-7.pdf

MECA500 Programming Manual: *https://cdn.mecademic.com/uploads/docs/meca500-r3-programming-manual-7.pdf*

Simulation Resolved Rates Code:

```
% this function performs the actual resolved rates algorithm based on our
% class lecture notes
% inputs: [starting joint values, desired position vector, desired rotation
% matrix, mecaVisualizer object, video object]
% outputs: [boolean- 1 if singularity offset is needed 0 otherwise
% (desired pose is reached), final joint values, final position vector,
% final rotation matrix]
function [crash, q, p_final, r_final] = resolved_rates(start_q, p_des, r_des,
meca, m)
%% setup poses values
crash = 0;
% robot joint conditions
joint_min = pi/180*[-175; -160; -135; -170; -115; -36000];
joint_max = pi/180*[175; 0; 70; 70; 115; 36000];
joint_vel = pi/180*[150; 150; 180; 300; 300; 500]; %max joint speeds
qmin_v = 0.5; %minimum joint velocity
% end-effector linear velocity limits
vmax = 5;
vmin = 0.1;
% end-effector angular velocity limits
omax = 2*pi;
omin = pi/180;
p_epsilon = .005; %maximum position error
r_epsilon = 0.1; %maximum rotation error
dt = 0.01;
%% start robot pose
meca.setJointAngles(start_q); % set joint angles
meca.plotRobot(); % update plot
frames = meca.getFrames(); %sets DH matrix/homogeneous transforms
%% declares current pose values/errors
p_cur = frames(1:3,4,6);
p_start = p_cur;
q_cur = start_q;
p_error = p_des-p_cur;
r_cur = frames(1:3, 1:3, 6);
r_error = r_des*r_cur';
err_axis=1/2*[r_error(3,2)-r_error(2,3);r_error(1,3)-
r_error(3,1);r_error(2,1)-r_error(1,2)];
tr = trace(r_error);
c_theta = (tr-1)/2;
s_theta=norm(err_axis);
theta = atan2(s_theta,c_theta);
%% collision offsets
if ((p_cur(2) > 0 && p_des(2) < 0)) || ((p_des(2) > 0 && p_cur(2) < 0))
   q_cur = q_offset(q_cur, pi/180*[0;-90;0;10;10;0], meca, m);
   meca.setJointAngles(q_cur);
   frames = meca.getFrames();
   p_cur = frames(1:3,4,6);
   p_error= p_des-p_cur;
elseif (p_cur(1) > 0 && p_des(1) < 0) || (p_des(1) > 0 && p_cur(1) < 0)
   q_cur = q_offset(q_cur, pi/180*[90;-90;0;10;10;0], meca, m);
   meca.setJointAngles(q_cur);
   frames = meca.getFrames();
   p_cur = frames(1:3,4,6);
```

```
    p_error= p_des-p_cur;
end
l = 0;
%% resolved rates loop
while norm(p_error) > p_epsilon || theta > r_epsilon
    % checks joint bounds
    if sum(q_cur == joint_max) > 0 || sum(q_cur == joint_min) > 0
        crash = 1;
        q = q_cur;
        p_final = p_cur;
        r_final = r_cur;
        return;
    end
    % adjust end effector velocities
    if norm(p_error) > p_epsilon*vmax/vmin
        vfactor = 1;
    elseif norm(p_error) < p_epsilon
        vfactor = 0;
    else
        vfactor = norm(p_error)/(vmax*p_epsilon/vmin);
    end
    velocity=vfactor*vmax*p_error/norm(p_error);
    if abs(theta) > r_epsilon*omax/omin
        ofactor = 1;
    elseif abs(theta) < r_epsilon
        ofactor=0;
    else
        ofactor=theta/(omax*r_epsilon/omin);
    end
    omega=ofactor*omax*err_axis;

    %calculate twist
    x_dot = real([velocity; omega]);
    frames = meca.getFrames();
    J = getJacobian(frames);

    %checks for singularity
    if l > 10 && round(det(J), 4) == 0
        disp("the robot has reached singularity");
        q = q_cur;
        p_final = p_cur;
        r_final = r_cur;
        crash = 1;
        return
    end

    % calculate joint velocities
    q_dot = pinv(J)*x_dot;
    % check against joint max speeds
    max_qscalar = max(abs(q_dot./joint_vel));
    if max_qscalar > 1
        q_dot = q_dot/max_qscalar;
    end
    if norm(q_dot) < qmin_v
        q_dot = qmin_v*q_dot./norm(q_dot);
    end
    % adjusts joints according to joint bounds
```

```
    q_cur = q_cur + q_dot*dt;
    for joints = 1:6
        if q_cur(joints) > joint_max(joints)
            q_cur(joints) = joint_max(joints);
        elseif q_cur(joints) < joint_min(joints)
            q_cur(joints) = joint_min(joints);
        end
    end
    meca.setJointAngles(q_cur); % set joint angles
    meca.plotRobot(); % update plot

    % Print ee position
    frames = meca.getFrames();
    % disp('EE position: ');
    % disp(frames(1:3,4,6));
    p_cur = frames(1:3,4,6);

    % recalculates pose values/error based on new joint values
    r_cur = frames(1:3, 1:3, 6);
    r_error = r_des*r_cur';
    err_axis=1/2*[r_error(3,2)-r_error(2,3);r_error(1,3)-
r_error(3,1);r_error(2,1)-r_error(1,2)];
    tr = trace(r_error);
    c_theta = (tr-1)/2;
    s_theta=norm(err_axis);
    theta = atan2(s_theta,c_theta);
    p_error = p_des - p_cur;

    % pause for visibility, and get video frame
    pause(0.01)
    l = l + 1;
    frame = getframe(1);
    writeVideo(m, frame);
end
q = q_cur;
p_final = p_cur;
r_final = r_cur;
end
```

GUI Resolved Rates Code:

```
function [crash, q, p_final, p_start] = resolvedRatesMeca(p_des, jointArray,
mecaRobotClient, crash_tol, correct)
%% setup poses values
crash = 0;
joint_min = pi/180*[-175; -70; -135; -170; -115; -36000]; %% Joint Limits
joint_max = pi/180*[175; 90; 170; 70; 115; 36000];
joint_vel = pi/180*[150; 150; 180; 300; 300; 500]; %% Joint Velocity Limits
qmin_v = 1;
vmax = 1;
vmin = 0.01;
omax = 2*pi;
omin = pi/180;
p_epsilon = .005;  %% Position error limit
```

```
r_epsilon = 0.1;   %% Rotation error limit
jointArray = jointArray(2:7)*pi/180;  %% Starting Joint Values

%% Set DH Matrix
DH_matrix=set_DH_matrix(jointArray');



r_des = rotd([1; 0; 0], p_des(4)) * rotd([0; 1; 0], p_des(5)) * rotd([0; 0; 1],
p_des(6)); %desired orientation matrix
p_epsilon = 0.005; %max position error
r_epsilon = 0.1; %max rotation error
dt = 0.1; %% Time Interval
p_des = p_des(1:3);  %% Origin of end pose in world frame
%% start robot pose

p_cur = getPose(mecaRobotClient,0);  %% Current Pose

frames = direct_kinematics_using_DH(DH_matrix);


pose = getPose(mecaRobotClient,1);
p_start = p_cur;
q_cur = jointArray';
p_error= p_des-p_cur;


%% check if 180 turn
%-----------------------------------------------------------------
%% Inversed Y-values
if (p_cur(2) > 0 && p_des(2) < 0) || (p_des(2) > 0 && p_cur(2) < 0)
    mecaRobotClient.run('MovePose',[200,0,285,-180,90,180])
    mecaRobotClient.getResponseMessage();

    p_cur = getPose(mecaRobotClient,0);

    q_cur = getJoints(mecaRobotClient);
    DH_matrix=set_DH_matrix(q_cur);
    frames = direct_kinematics_using_DH(DH_matrix);
    q_cur = q_cur';

    p_error= p_des-p_cur;
end

%% Inversed X-values
if (p_cur(1) > 0 && p_des(1) < 0) || (p_des(1) > 0 && p_cur(1) < 0)
    if p_des(2) > 0
    mecaRobotClient.run('MoveJoints',[90,0,0,10,10,0])
    mecaRobotClient.getResponseMessage();

    p_cur = getPose(mecaRobotClient,0);

    q_cur = getJoints(mecaRobotClient);
    DH_matrix=set_DH_matrix(q_cur);
    frames = direct_kinematics_using_DH(DH_matrix);
```

```
    q_cur = q_cur';

    p_error= p_des-p_cur;
    else
    mecaRobotClient.run('MoveJoints',[90,0,0,10,10,0])
    mecaRobotClient.getResponseMessage();

    p_cur = getPose(mecaRobotClient,0);

    q_cur = getJoints(mecaRobotClient);
    DH_matrix=set_DH_matrix(q_cur);
    frames = direct_kinematics_using_DH(DH_matrix);
    q_cur = q_cur';

    p_error= p_des-p_cur;

    end
end
%---------------------------------------------------------------------
%---------------------------------------------------------------------


r_cur = frames(1:3, 1:3, 6);
r_error = r_des*r_cur';
err_axis=1/2*[r_error(3,2)-r_error(2,3);r_error(1,3)-r_error(3,1);r_error(2,1)-
r_error(1,2)];
tr = trace(r_error);
c_theta = (tr-1)/2;
s_theta=norm(err_axis);
theta = atan2(s_theta,c_theta);
l = 0;


while norm(p_error) > p_epsilon || theta > r_epsilon
    singular = 0;

    J = getJacobian(frames);  % Obtain current Jacobian


    %% Account for Joint Limits, and offset
    if sum(q_cur == joint_max) > 0 || sum(q_cur == joint_min) > 0
        h = q_cur .* [1,1,1,-0.5,-1,-0.25]';

        mecaRobotClient.run('MoveJoints',(h/pi*180)');
        mecaRobotClient.getResponseMessage();

        q_cur = getJoints(mecaRobotClient);
        DH_matrix=set_DH_matrix(q_cur);
        frames = direct_kinematics_using_DH(DH_matrix);
        q_cur = q_cur';

        p_cur = getPose(mecaRobotClient,0);

        r_cur = frames(1:3, 1:3, 6);
        r_error = r_des*r_cur';
```

```
        err_axis=1/2*[r_error(3,2)-r_error(2,3);r_error(1,3)-
r_error(3,1);r_error(2,1)-r_error(1,2)];
        tr = trace(r_error);
        c_theta = (tr-1)/2;
        s_theta=norm(err_axis);
        theta = atan2(s_theta,c_theta);
    end
    %%-------------------------------------------------------------------


    %% Compute Instantaneous position change
    if norm(p_error) > p_epsilon*vmax/vmin || correct == 2
        vfactor = 1;
    elseif norm(p_error) < p_epsilon
        vfactor = 0;
    else
        vfactor = norm(p_error)/(vmax*p_epsilon/vmin);
    end
    velocity=vfactor*vmax*p_error(1:3)/norm(p_error(1:3));

    if abs(theta) > r_epsilon*omax/omin || correct == 2
        ofactor = 1;
    elseif abs(theta) < r_epsilon
        ofactor=0;
    else
        ofactor=theta/(omax*r_epsilon/omin);
    end
    omega=ofactor*omax*err_axis;

    x_dot = real([velocity; omega]);
    %%-------------------------------------------------------------------


    if l > 10 && round(det(J), 5) == 0
        disp("the robot has reached singularity");
        q = q_cur;
        p_final = p_cur;
        r_final = r_cur;
        return
    end

    disp(round(det(J), 4));
    q_dot = pinv(J)*x_dot;

    % check if q > robot max values// q_dot > max speeds...
    max_qscalar = max(abs(q_dot./joint_vel));
    if max_qscalar > 1
        q_dot = q_dot/max_qscalar;
    end
    if norm(q_dot) < qmin_v
        q_dot = qmin_v*q_dot./norm(q_dot);
    end

    disp(norm(q_dot));
    q_cur = q_cur + q_dot*dt;
```

```
    % check joint max values
    for joints = 1:6
        if q_cur(joints) > joint_max(joints)
            q_cur(joints) = joint_max(joints);
        elseif q_cur(joints) < joint_min(joints)
            q_cur(joints) = joint_min(joints);
        end
    end

    %% Move Joints to new values
    DHMatrix = set_DH_matrix(q_cur);
    mecaRobotClient.run('MoveJoints',(q_cur*180/pi)');
    mecaRobotClient.getResponseMessage();


    % Print ee position
    frames = direct_kinematics_using_DH(DHMatrix);
    disp('EE position: ');
    disp(frames(1:3,4,6));
    p_cur = getPose(mecaRobotClient,0);


    % ee orientation
    r_cur = frames(1:3, 1:3, 6);
    r_error = r_des*r_cur';
    err_axis=1/2*[r_error(3,2)-r_error(2,3);r_error(1,3)-r_error(3,1);r_error(2,1)-
r_error(1,2)];
    tr = trace(r_error);
    c_theta = (tr-1)/2;
    s_theta=norm(err_axis);
    theta = atan2(s_theta,c_theta);
    if singular == 0
        p_error = p_des - p_cur;
    end
    pause(0.01)
    l = l + 1;
    if correct == 1 && l > 15
        q = q_cur;
        p_final = p_cur;
        r_final = r_cur;
        return
    end
end
q = q_cur;
p_final = p_cur;
r_final = r_cur;
end
```